1 Coordinate Frames and Time Systems

Every model in this reference — the tropospheric and ionospheric mappings of the Atmosphere document, the solid-Earth, ocean, and pole tides of the Site Displacement document, the antenna, attitude, and phase-wind-up terms of the Observation Physics document, and the orbit evaluation of the Product Evaluation document — presumes a fixed geodetic-frame and time-scale substrate. This document specifies that substrate: the WGS84 ellipsoid and the three coordinate representations (Cartesian ECEF, geodetic, and local ENU) the engine moves between; the family of GNSS time scales together with the leap- second machinery that ties them to UTC; and the Earth-orientation chain that rotates the celestial frame — in which the Sun, Moon, and the satellite ephemerides are most naturally expressed — into the terrestrial frame in which the receiver sits. None of the conversions here is an estimator state. The one frame quantity that *is* estimated — the inter-system time offset (ISB) that absorbs the unmodelled bias between constellation time scales — belongs to the State Vector document; this document covers only the *deterministic* scale conversions that the ISB state is defined relative to. Code references are to `src/core/math/Coordinates.cpp`, `src/core/math/TimeUtils.cpp`, `src/core/model/Planetary.cpp`, `src/core/products/LiveEphemerisStore.cpp`, and `src/core/products/ProductParser.cpp`.

1.1 The WGS84 ellipsoid: ECEF, geodetic, and local frames

The terrestrial reference frame is the WGS84 Earth-Centred, Earth-Fixed (ECEF) Cartesian system: origin at the geocentre, z along the conventional pole, x toward the IERS reference meridian, y completing a right-handed triad. The defining ellipsoid has semi-major axis a and flattening f, from which the first eccentricity squared follows (`src/core/math/Coordinates.cpp:11`, :55):

$$e^2 \;=\; f\,(2-f). \tag{1}$$

The radius of curvature in the prime vertical at geodetic latitude φ is

$$N(\varphi) \;=\; \frac{a}{\sqrt{1-e^2\,\sin^2\varphi}}, \tag{2}$$

which scales the entire forward map from geodetic coordinates (φ, λ, h) to ECEF (`src/core/math/Coordinates.cpp:54--68`):

$$\boxed{\begin{aligned} x &= \big(N+h\big)\cos\varphi\,\cos\lambda, \\ y &= \big(N+h\big)\cos\varphi\,\sin\lambda, \\ z &= \big((1-e^2)\,N+h\big)\sin\varphi. \end{aligned}} \tag{3}$$

The factor $(1-e^2)N$ on the z component is the source of all ellipsoidal subtlety: a sphere would carry N unmodified on every axis, and the $(1-e^2)$ correction is precisely the contraction that distinguishes the geodetic normal (perpendicular to the ellipsoid surface) from the geocentric radius vector.

Inverse map (Bowring fixed point). The inverse problem — recovering (φ, λ, h) from (x, y, z) — has no closed form for the latitude because N in (3) itself depends on φ. The longitude is immediate,

$\lambda = \operatorname{atan2}(y, x)$. For the latitude the engine runs a Bowring fixed-point iteration on the equatorial projection $p = \sqrt{x^2 + y^2}$ (`src/core/math/Coordinates.cpp:21--47`). The seed neglects height,

$$\varphi^{(0)} = \operatorname{atan2}\big(z,\ p\,(1 - e^2)\big), \tag{4}$$

and each pass recomputes N and the height and re-evaluates the latitude with the height feedback folded back into the eccentricity term:

$$h^{(k)} = \frac{p}{\cos\varphi^{(k)}} - N^{(k)}, \qquad \varphi^{(k+1)} = \operatorname{atan2}\left(z,\ p\left[1 - e^2\,\frac{N^{(k)}}{N^{(k)} + h^{(k)}}\right]\right). \tag{5}$$

The iteration is capped at ten passes and exits early once $|\varphi^{(k+1)} - \varphi^{(k)}| < 10^{-13}$; for terrestrial heights it converges to machine precision in two or three steps.

Pole branch. When $p < 10^{-12}\,\mathrm{m}$ the point lies on (or within a picometre of) the z axis and both the longitude and the $p/\cos\varphi$ height formula degenerate. The code takes an explicit branch (`Coordinates::ecefToLlh`, the $p < 10^{-12}$ guard): the longitude retains the $\operatorname{atan2}(y, x)$ value computed before the guard — formally undefined on the axis, and immaterial there — while the branch fixes $\varphi = \pm\pi/2$ by the sign of z and reads the height directly off the polar semi-minor axis,

$$h = |z| - N\,(1 - e^2) = |z| - a\sqrt{1 - e^2} \quad (\text{at the pole},\ \sin^2\varphi = 1), \tag{6}$$

since $N(1 - e^2)$ evaluated at the pole is exactly the polar radius $a\sqrt{1 - e^2}$. This guard keeps the inverse map total over all of $\mathbb{R}^3$, which matters because the planetary and attitude code occasionally evaluates geometry for satellite positions that transit near the polar axis.

Local East–North–Up frame. Many of the downstream models — the tropospheric mapping function, the elevation mask, the antenna phase-centre projection — are most natural in the topocentric ENU frame anchored at the receiver's geodetic (φ, λ). The columns of the ENU→ECEF rotation are the three local unit vectors expressed in ECEF (`src/core/math/Coordinates.cpp:70--81`):

$$\boxed{\boldsymbol{R}_{\mathrm{ENU}}(\varphi, \lambda) = \begin{bmatrix} \hat{\boldsymbol{e}}_{\mathrm{E}} & \hat{\boldsymbol{e}}_{\mathrm{N}} & \hat{\boldsymbol{e}}_{\mathrm{U}} \end{bmatrix} = \begin{bmatrix} -\sin\lambda & -\sin\varphi\cos\lambda & \cos\varphi\cos\lambda \\ \cos\lambda & -\sin\varphi\sin\lambda & \cos\varphi\sin\lambda \\ 0 & \cos\varphi & \sin\varphi \end{bmatrix}.} \tag{7}$$

The Up column is the outward geodetic normal — identical to the direction along which h is measured in (3) — East is its horizontal cross-product with the pole, and North completes the right-handed triad. Because the columns are orthonormal, $\boldsymbol{R}_{\mathrm{ENU}}$ is a proper rotation and its inverse is its transpose: an ECEF displacement is resolved into local components by $\boldsymbol{v}_{\mathrm{ENU}} = \boldsymbol{R}_{\mathrm{ENU}}^{\mathrm{T}}\,\boldsymbol{v}_{\mathrm{ECEF}}$ (`src/core/math/Coordinates.cpp:83--92`), avoiding any explicit matrix inversion.

Elevation and azimuth. With the receiver-to-satellite line-of-sight resolved into the local frame, $\boldsymbol{e}_{\mathrm{ENU}} = (e_{\mathrm{E}}, e_{\mathrm{N}}, e_{\mathrm{U}})^{\mathrm{T}} = \boldsymbol{R}_{\mathrm{ENU}}^{\mathrm{T}}\boldsymbol{e}$, the elevation and azimuth that gate the measurement (the elevation mask of the Observation Equations document and the mapping-function argument of the Atmosphere document) follow directly:

$$\mathrm{el} = \arcsin\big(e_{\mathrm{U}}\big), \qquad \mathrm{az} = \operatorname{atan2}\big(e_{\mathrm{E}},\ e_{\mathrm{N}}\big), \tag{8}$$

azimuth measured clockwise from North. Equations (7)–(8) are the entire geometric bridge between the ECEF state vector and every elevation-dependent weighting and antenna model in the reference.

1.2 GNSS time scales and the GPS reference epoch

Time inside the engine is carried as a `GnssTime` $\{\text{week}, \text{tow}\}$ pair — an integer GPS week and a floating-point time-of-week in seconds — and is expanded to a single continuous count $T = \text{week} \times 604800 + \text{tow}$ wherever a monotone scalar is needed (e.g. ERP interpolation, Section 1.8). A week boundary is 604 800 s and a day 86 400 s (`src/core/math/TimeUtils.cpp`, header constants `SECONDS_IN_WEEK`, `SECONDS_IN_DAY`). The common origin is the GPS epoch, 1980-01-06 00:00:00 UTC, whose Julian Date is fixed in the code:

$$\text{JD}_{\text{GPS}} = 2444244.5 \qquad (\texttt{src/core/math/TimeUtils.cpp}, \texttt{GPS_EPOCH_JD}). \tag{9}$$

GPS time is a continuous atomic scale — it never inserts a leap second — so it sits a fixed integer offset below International Atomic Time and a fixed fractional offset below Terrestrial Time (`src/core/math/TimeUtils.cpp`, header constants `TAI_GPS_OFFSET`, `TT_GPS_OFFSET`):

$$\boxed{\text{TAI} - \text{GPS} = 19 \text{ s}, \qquad \text{TT} - \text{GPS} = 51.184 \text{ s} = (32.184 + 19) \text{ s}.} \tag{10}$$

The 32.184 s in the TT relation is the constant TT−TAI offset; together these let any GPS time be promoted to the TT scale on which precession and nutation are evaluated. Terrestrial Time relative to the J2000.0 epoch is measured in Julian centuries,

$$t = \frac{\text{JD}_{\text{TT}} - 2451545.0}{36525.0}, \tag{11}$$

the dimensionless argument of the astronomical series of Sections 1.6 and 1.9 (`TimeUtils::jdToJ2000Centuries`, constants `J2000_JD`, `DAYS_PER_JULIAN_CENTURY`).

Gregorian↔Julian Date. The calendar conversions follow the Meeus algorithm [2]. The forward map, with the month shifted so January and February count as months 13 and 14 of the prior year and a Gregorian-reform correction $b = 2 - A + \lfloor A/4 \rfloor$ for $A = \lfloor y/100 \rfloor$, is (`TimeUtils::gregorianToJulianDay`):

$$\text{JD} = \lfloor 365.25\,(y + 4716) \rfloor + \lfloor 30.6001\,(m + 1) \rfloor + d + b - 1524.5 + \frac{(\text{time of day})}{86400}, \tag{12}$$

and the inverse (`TimeUtils::julianDayToGregorian`) reverses it through the integer auxiliaries $z = \lfloor \text{JD} + 0.5 \rfloor$ and the Gregorian-reform branch at $z \geq 2299161$. The crucial distinction the code maintains throughout is between a calendar date that is already on the GPS scale (no leap correction: `gregorianGpsToGpsTime`) and one that is on UTC and must have the epoch's leap seconds added back to land on GPS time (`TimeUtils::gregorianToGpsTime`); conflating the two is the classic source of whole-second timing errors.

1.3 Leap seconds and per-system dispatch

UTC is tied to GPS by the historical table of leap-second insertions. The engine stores the table as $\big(\text{JD}_k,\ \Delta_{\text{GU}}(k)\big)$ pairs — the GPS−UTC offset that takes effect at each insertion boundary, the first row being the GPS epoch itself with offset zero (`src/core/math/TimeUtils.cpp`, `LEAP_TABLE`). A query resolves the offset by a reverse scan that returns the offset of the latest boundary not after the epoch:

$$\Delta_{\text{GU}}(\text{JD}) = \Delta_{\text{GU}}(k^\star), \qquad k^\star = \max\{\, k : \text{JD}_k \leq \text{JD} \,\}, \tag{13}$$

falling back to zero before the epoch (`TimeUtils::leapSecondsGpsUtc`). TAI−UTC is then simply $\Delta_{\text{GU}} + 19$ s from (10) (`TimeUtils::leapSecondsTaiUtc`). The epoch-dependence is essential: the same UTC calendar instant maps to different GPS times in 2005 and 2010 because a leap second was inserted between them, and only a boundary-aware lookup recovers the correct offset.

The wall-clock leap constant and its runtime override. The boundary-aware table of (13) is the authority for every *epoch-referenced* conversion — a dated calendar timestamp, GLONASS day arithmetic, UTC formatting — because each of those carries the epoch at which the correct offset is known. The one conversion that does not is the promotion of the *system wall clock* to GPS time (`TimeUtils::nowContinuousGpsSeconds`, `nowGpsTime`), used only on the real-time decode and latency-stamping paths: POSIX `system_clock` is UTC and carries no epoch of its own, so the offset cannot be looked up and must instead be supplied as a fixed count. That count is the compiled constant `TimeUtils::LEAP_SECONDS` = 18 s (valid from 2017-01-01), read through an atomic accessor (`currentLeapSeconds`) so that a future insertion can be applied to a live binary without a rebuild: `setLeapSecondsOverride`, driven by the config key `realtime.leap_seconds` (`src/core/config/GnssConfigLoad.cpp`), replaces it at real-time startup. The override deliberately touches *only* the wall-clock path; the epoch-dependent (13) lookups are left alone, since they are already correct at every historical epoch and an override there would corrupt rather than update them.

Per-constellation Gregorian dispatch. The four supported time systems differ only in how a RINEX/broadcast calendar timestamp is referred to the GPS scale, dispatched by a system tag (`TimeUtils::GnssTimeFromDate`):

- **GPS / Galileo** — already on the GPS scale; the timestamp is converted with no leap correction (GST is steered to GPS time and shares the same week/second framing to within the few nanoseconds the ISB state of the State Vector document absorbs).
- **BeiDou** — the timestamp is on BDT, which runs 14 s behind GPS (Section 1.5); the code converts on the GPS-scale assumption and then *adds* 14 s, carrying any week roll-over (`TimeUtils::GnssTimeFromDate`, BeiDou branch).
- **GLONASS** — the RINEX timestamp is UTC; the full leap correction of (13) is applied to bring it onto GPS time (`TimeUtils::GnssTimeFromDate`, GLONASS branch).

GPS and Galileo therefore carry no leap correction, BeiDou a fixed 14 s, and GLONASS the full epoch-dependent leap count — a dispatch that exactly mirrors the constellations' defining time references.

1.4 GLONASS time-of-day to full GPS week

GLONASS broadcast and SSR records frequently carry only a time-of-day — seconds since the start of the GLONASS day — with no week and no absolute date. GLONASS time runs on UTC(SU) advanced by three hours (Moscow), so its day boundary is three hours ahead of the UTC midnight. Reconstructing a full GPS week from a bare time-of-day therefore needs an *anchor*: an approximate GPS-seconds estimate of the true epoch (typically the current obs epoch), against which the day and the ±12 h wrap are resolved (`TimeUtils::glonassTodToGpsTime`, after RTKLIB's `adjday_glot`). The procedure, with day length $D = 86\,400$ s, half-day $D/2$, and Moscow offset $M = 3 \times 3600$ s:

1. Convert the anchor GPS seconds to UTC by subtracting the leap count at that epoch, $t_{\mathrm{UTC}} = t_{\mathrm{anchor}} - \Delta_{\mathrm{GU}}$, then shift into the Moscow-day frame $t_{\mathrm{M}} = t_{\mathrm{UTC}} + M$ and locate that day's midnight $t_0 = \lfloor t_{\mathrm{M}}/D \rfloor\, D$, giving the anchor's time-of-day $\tau_{\mathrm{a}} = t_{\mathrm{M}} - t_0$.
2. Place the reported time-of-day τ in the anchor's day, resolving the midnight wrap toward the nearest match:

$$\boxed{\tau \leftarrow \begin{cases} \tau + D, & \tau < \tau_{\mathrm{a}} - D/2, \\ \tau - D, & \tau > \tau_{\mathrm{a}} + D/2, \\ \tau, & \text{otherwise}, \end{cases} \qquad t_{\mathrm{GPS}} = \underbrace{(t_0 + \tau)}_{\text{Moscow}} - M + \Delta_{\mathrm{GU}}.} \tag{14}$$

The final line undoes the Moscow offset to recover UTC and then *re-adds the same leap count* Δ_{GU} to climb back onto the GPS scale. The leap term enters once with each sign and so cancels exactly between the two legs — it is present only because the intermediate arithmetic must pass through UTC to frame the Moscow day correctly. If no usable anchor is supplied (a non-positive value) the routine returns the sentinel $\{0, 0\}$ rather than fabricate a week, so the caller can detect the failure and fall back; the continuous-seconds-to-week split that closes the routine carries the standard negative-TOW correction (`gpsSecondsToGnssTime`).

1.5 BeiDou time: the 14 s offset and independent week base

BeiDou time (BDT) is offset from GPS by a sub-second-level integer and by an independent week numbering, and the two pieces are owned by different modules. The *sub-second* relation,

$$\mathrm{BDT} = \mathrm{GPS} - 14\ \mathrm{s}, \tag{15}$$

reflects that BDT was synchronised to UTC at its 2006 zero epoch, by which time GPS already led UTC by 14 s; it is applied in `TimeUtils` when a BeiDou calendar timestamp is converted to GPS time (`TimeUtils::GnssTimeFromDate`, BeiDou branch; Section 1.3). The *week* relation is separate. BDT counts weeks from its own zero epoch (2006-01-01), which is GPS week 1356, and this base is applied at query time by the ephemeris store, not by `TimeUtils` (`src/core/products/LiveEphemerisStore.cpp:29`, :442). When the store lifts a BeiDou ephemeris reference time onto the common GPS scale for an honest cross-system age check it adds the combined offset

$$\boxed{\Delta_{\mathrm{BDT}\to\mathrm{GPS}} = 1356 \times 604800\ \mathrm{s} + 14\ \mathrm{s},} \tag{16}$$

the week base plus the 14 s of (15) (`src/core/products/LiveEphemerisStore.cpp:29--39`). Omitting the week base would make every BeiDou satellite read roughly 1356 weeks ($\sim$26 years) stale and be silently rejected by the staleness guard that follows; the offset does not cancel automatically because the query time is GPS-scale while the stored t_{oe} is on the native BDT week (`src/core/products/LiveEphemerisStore.cpp:438--448`).

1.6 Legacy ECI→ECEF: the equinox-based chain

The Sun, Moon, and (for some product streams) the satellite positions are most naturally given in a quasi-inertial celestial frame; the tides, attitude, and phase wind-up that consume them live in the terrestrial ECEF frame. The bridge is the Earth-orientation matrix $\boldsymbol{U}$ such that $\boldsymbol{r}_{\mathrm{ECEF}} = \boldsymbol{U}\,\boldsymbol{r}_{\mathrm{ECI}}$. The default (always-available) path is the classical equinox-based chain following the IAU-1976 precession and IAU-1980 nutation conventions [4], composed as (`src/core/model/Planetary.cpp:356--397`):

$$\boxed{\boldsymbol{U} = \boldsymbol{W}_{\mathrm{pm}}\ \boldsymbol{R}_3(\mathrm{GAST})\ \boldsymbol{N}_{\mathrm{nut}}\ \boldsymbol{P}_{\mathrm{prec}}.} \tag{17}$$

Here $\boldsymbol{R}_1, \boldsymbol{R}_2, \boldsymbol{R}_3$ are the elementary right-handed axis rotations of `Planetary::Rxyz` (`Planetary.cpp:50--67`) and each factor reverses one piece of the celestial-to-terrestrial orientation: precession and nutation orient the true equator and equinox of date, the sidereal rotation $\boldsymbol{R}_3(\mathrm{GAST})$ spins the frame about the pole by the apparent sidereal time, and polar motion $\boldsymbol{W}_{\mathrm{pm}}$ ties the celestial pole to the conventional terrestrial pole.

Precession (IAU 1976). With t the TT centuries of (11) and arcsecond coefficients converted to radians, the three equatorial precession angles ζ_A, θ_A, z_A and the mean obliquity ε_0 are (`src/core/model/Planetary.cpp:374--379`):

$$\begin{aligned} \zeta_A &= 2306.2181\,t + 0.30188\,t^2 + 0.017998\,t^3, \\ \theta_A &= 2004.3109\,t - 0.42665\,t^2 - 0.041833\,t^3, \\ z_A &= 2306.2181\,t + 1.09468\,t^2 + 0.018203\,t^3, \\ \varepsilon_0 &= 84381.448 - 46.8150\,t - 0.00059\,t^2 + 0.001813\,t^3 \quad \text{(arcsec)}, \end{aligned} \tag{18}$$

assembled as $\boldsymbol{P}_{\text{prec}} = \boldsymbol{R}_3(-z_A)\,\boldsymbol{R}_2(\theta_A)\,\boldsymbol{R}_3(-\zeta_A)$.

Nutation (IAU 1980, 106 terms). The nutation in longitude $\Delta\psi$ and in obliquity $\Delta\varepsilon$ are summed over the 106-term IAU-1980 series [4] in the five Delaunay fundamental arguments $f = (l, l', F, D, \Omega)$ (`src/core/model/Planetary.cpp:96--231`; arguments from `astArgs`, `Planetary.cpp:70--94`):

$$\Delta\psi = 10^{-4}\sum_{i=1}^{106}\big(A_i^{\psi} + \dot{A}_i^{\psi}\,t\big)\sin\big(\boldsymbol{n}_i\cdot f\big), \quad \Delta\varepsilon = 10^{-4}\sum_{i=1}^{106}\big(A_i^{\varepsilon} + \dot{A}_i^{\varepsilon}\,t\big)\cos\big(\boldsymbol{n}_i\cdot f\big), \tag{19}$$

in arcseconds (the 10^{-4} scales the tabulated 0.1 mas units), then converted to radians. The nutation matrix rotates by the true obliquity $\varepsilon_0 + \Delta\varepsilon$: $\boldsymbol{N}_{\text{nut}} = \boldsymbol{R}_1(-\varepsilon_0 - \Delta\varepsilon)\,\boldsymbol{R}_3(-\Delta\psi)\,\boldsymbol{R}_1(\varepsilon_0)$ (`Planetary.cpp:381--384`).

Apparent sidereal time and the 0h-UT1 subtlety. Greenwich Apparent Sidereal Time adds the equation of the equinoxes — the nutation in longitude projected on the equator, plus the two leading complementary terms in the lunar node Ω — to the mean sidereal time (`src/core/model/Planetary.cpp:386--388`):

$$\text{GAST} = \text{GMST} + \Delta\psi\cos\varepsilon_0 + \big(0.00264\sin\Omega + 0.000063\sin 2\Omega\big)\ \text{arcsec}. \tag{20}$$

GMST itself is built from the IAU-1982 model evaluated at the *start* of the current UT1 day plus the Earth's rotation through the elapsed day fraction (`Planetary.cpp:242--263`). The reference instant must be 0h UT1 of the calendar day, and because a Julian Date is referenced to *noon*, that midnight is recovered as

$$\text{JD}_{0\text{h}} = \big\lfloor \text{JD}_{\text{UT1}} - 0.5 \big\rfloor + 0.5, \tag{21}$$

not $\lfloor \text{JD} \rfloor$ (`Planetary.cpp:245--251`). Using the bare floor lands on the *previous* noon for any post-midnight epoch — a half-day error that propagates to roughly 0.5° in GMST and, through (17), on the order of 10^6 km in the rotated Sun position; the comment at the cited line records exactly this failure mode.

Polar motion. The terrestrial-pole tie is $\boldsymbol{W}_{\text{pm}} = \boldsymbol{R}_2(-x_p)\,\boldsymbol{R}_1(-y_p)$ for the interpolated pole coordinates (x_p, y_p) in radians (`src/core/model/Planetary.cpp:390--392`), the small-angle two-axis rotation that aligns the celestial intermediate pole of date with the IERS conventional pole.

1.7 CIO-based ITRS↔GCRS (IAU 2000A/2006)

An opt-in path (`enable_cio_frame_`) replaces the equinox chain with the modern Celestial Intermediate Origin (CIO) based transform of the IAU 2000A/2006 conventions [4], evaluated through SOFA [1] and mirroring the M-PAGES `NgsFormC2T` construction (`src/core/model/Planetary.cpp:399--446`). The celestial-to-terrestrial matrix is composed as

$$\boxed{\boldsymbol{U} = \mathtt{POM}(x_p, y_p, s')\ \boldsymbol{R}_3(\theta_{\mathrm{ERA}})\ \mathtt{C2I}(X, Y, s),} \tag{22}$$

i.e. the SOFA composition `iauC2tcio`. The Celestial Intermediate Pole coordinates (X, Y) and the CIO locator s come from the full IAU-2000A precession-nutation model via `iauXys00a`, after which the *observed* celestial-pole offsets are injected additively,

$$X \leftarrow X + \mathrm{d}X, \qquad Y \leftarrow Y + \mathrm{d}Y, \tag{23}$$

and the celestial-to-intermediate matrix is formed by `iauC2ixys`. The diurnal rotation is the Earth Rotation Angle θ_{ERA} (a strictly linear function of UT1, `iauEra00`) rather than the sidereal time of (20), and the terrestrial-pole tie `POM` uses `iauPom00` with the TIO locator s' from `iauSp00`. Two-part Julian Dates are passed with the (2400000.5, MJD) split for numerical conditioning (`Planetary.cpp:410--415`). The two paths are designed to agree: with $\mathrm{d}X = \mathrm{d}Y = 0$ the CIO result reproduces the bare IAU-2000A model and differs from the legacy equinox chain only by the (sub-milliarcsecond) frame-bias and model-version difference, which the engine confirms at startup by logging the angular separation of the two Sun directions (`Planetary.cpp:587--604`).

1.8 Earth-orientation parameters: parse and interpolation

Both transforms above need the time-varying Earth-orientation parameters $(x_p, y_p, \mathrm{UT1} - \mathrm{UTC}, \mathrm{d}X, \mathrm{d}Y)$. These arrive in a standard IGS `.ERP` file, parsed by `ProductParser::parseErp` (`src/core/products/ProductParser.cpp`). The parser reads the MJD time tag and the pole/UT1 columns and applies the IGS scale factors — the pole coordinates are tabulated in 10^{-6} arcseconds and UT1−UTC in 10^{-7} seconds (`ProductParser.cpp:1215--1217`):

$$x_p = (\mathrm{col}) \times 10^{-6}\,\mathrm{arcsec}, \quad y_p = (\mathrm{col}) \times 10^{-6}\,\mathrm{arcsec}, \quad (\mathrm{UT1} - \mathrm{UTC}) = (\mathrm{col}) \times 10^{-7}\,\mathrm{s}. \tag{24}$$

The MJD tag is converted to a GPS week/TOW relative to the GPS-epoch MJD (`ProductParser.cpp:1208--1211`). The CIP offsets $\mathrm{d}X, \mathrm{d}Y$ are *not* carried by the standard `.ERP` format and default to zero; because the IAU-2000A model of (22) already includes the modelled precession-nutation, $\mathrm{d}X = \mathrm{d}Y = 0$ is the correct nominal value, not a missing input (`ProductParser.cpp:1218--1221`). The parser *only* parses; all temporal interpolation lives in `Planetary::interpolateErp` (`src/core/model/Planetary.cpp:448--520`), which binary-searches the chronologically sorted table on continuous GPS seconds and linearly interpolates every component between the bracketing entries:

$$p(t) = p_0 + \frac{t - t_0}{t_1 - t_0}\,(p_1 - p_0), \qquad p \in \{x_p, y_p, \mathrm{UT1}{-}\mathrm{UTC}, \mathrm{d}X, \mathrm{d}Y\}, \tag{25}$$

clamping to the nearest endpoint outside the table span (`Planetary.cpp:486--519`). The interpolated (x_p, y_p) feed the polar-motion factor of (17) / (22), while UT1−UTC corrects UTC to UT1 before the sidereal/ERA evaluation. The product-evaluation document (Product Evaluation) cross-references this section rather than re-deriving the polar-motion application, since the same $\boldsymbol{W}_{\mathrm{pm}}$ rotates both the luni-solar vectors and any celestial-frame satellite product.

1.9 The Sun and Moon series these transforms position

The end purpose of the orientation chain is to place the Sun and Moon in ECEF for the geophysical and observation-model corrections. The celestial-frame positions are computed first, from an analytic series in the J2000 centuries t and the Delaunay arguments f (`src/core/model/Planetary.cpp:264--354`), an enhanced RTKLIB/Meeus model [2, 3]. The Sun's ecliptic longitude is the mean longitude plus the equation of centre (three harmonics of the solar mean anomaly $M_\odot$) and small Venus, Jupiter, and lunar perturbation terms (`Planetary.cpp:303--310`); the obliquity rotation

$$\varepsilon = 23.439291 - 0.0130042\,t \quad \text{(deg)} \tag{26}$$

(`Planetary.cpp:290`) carries the resulting ecliptic $(\lambda_\odot, \beta_\odot)$ into equatorial ECI components (`Planetary.cpp:324--331`):

$$\begin{aligned} r_x &= r\,\cos\beta_\odot \cos\lambda_\odot, \\ r_y &= r\,(\cos\varepsilon\,\cos\beta_\odot \sin\lambda_\odot - \sin\varepsilon\,\sin\beta_\odot), \\ r_z &= r\,(\sin\varepsilon\,\cos\beta_\odot \sin\lambda_\odot + \cos\varepsilon\,\sin\beta_\odot), \end{aligned} \tag{27}$$

with the solar distance r from the three-harmonic radius series (`Planetary.cpp:317--322`). The Moon's longitude, latitude, and horizontal-parallax-derived distance follow the analogous Brown-series truncation in the same five fundamental arguments (`Planetary.cpp:333--353`). The resulting ECI vectors are then rotated to ECEF by exactly the $\boldsymbol{U}$ of (17) or (22) according to the `enable_cio_frame_` toggle (`Planetary.cpp:535--612`); the ECEF Sun and Moon are precisely the inputs consumed by the solid-Earth and ocean tides of the Site Displacement document and by the satellite-attitude and phase-wind-up models of the Observation Physics document. The two luni-solar models are only ∼100 km accurate in position — ample for the tidal and attitude geometry, which depend on the Sun/Moon *direction* to well under the milliarcsecond level that matters — and a non-finite guard substitutes a nominal 1 AU / mean-lunar-distance fallback rather than propagate a NaN into the correction stack (`Planetary.cpp:614--622`).

References

[1] IAU SOFA Board. *SOFA Tools for Earth Attitude*. Software version 18; http://www.iausofa.org/. International Astronomical Union, Standards of Fundamental Astronomy (SOFA). 2021.

[2] Jean Meeus. *Astronomical Algorithms*. Richmond, VA, USA: Willmann-Bell, 1991. ISBN: 0-943396-35-2.

[3] Oliver Montenbruck and Eberhard Gill. *Satellite Orbits: Models, Methods and Applications*. Berlin, Heidelberg: Springer, 2000. DOI: 10.1007/978-3-642-58351-3.

[4] Gérard Petit and Brian Luzum. *IERS Conventions (2010)*. IERS Technical Note 36. International Earth Rotation and Reference Systems Service (IERS). Frankfurt am Main: Verlag des Bundesamts für Kartographie und Geodäsie, 2010. ISBN: 3-89888-989-6.